

Documentation Technique Complète -- Plateforme Sécurité Routière

Table des matières

- [Présentation du projet](#1-présentation-du-projet)
- [Architecture générale](#2-architecture-générale)
- [Stack technique](#3-stack-technique)
- [Structure du projet](#4-structure-du-projet)
- [Base de données -- Modèles](#5-base-de-données--modèles)
- [API Backend -- Endpoints](#6-api-backend--endpoints)
- [Services Backend](#7-services-backend)
- [Sécurité -- OWASP](#8-sécurité--owasp)
- [Frontend Angular](#9-frontend-angular)
- [Chatbot RAG -- Pipeline](#10-chatbot-rag--pipeline)
- [Détection de panneaux -- Vision IA](#11-détection-de-panneaux--vision-ia)
- [Fonctionnalités audio -- STT / TTS](#12-fonctionnalités-audio--stt--tts)
- [Rôles et interfaces utilisateurs](#13-rôles-et-interfaces-utilisateurs)
- [Commandes de démarrage](#14-commandes-de-démarrage)
- [Variables d'environnement](#15-variables-d'environnement)
- [Packages et dépendances](#16-packages-et-dépendances)

1. Présentation du projet

Nom : Plateforme de Sécurité Routière -- PFE

Objectif : Développer une plateforme web éducative pour l'apprentissage du code de la route en Tunisie, intégrant un chatbot intelligent bilingue (Français / Arabe) basé sur l'IA (RAG), un système de détection de panneaux par image, des cours, des QCM, et un tableau de bord statistique.

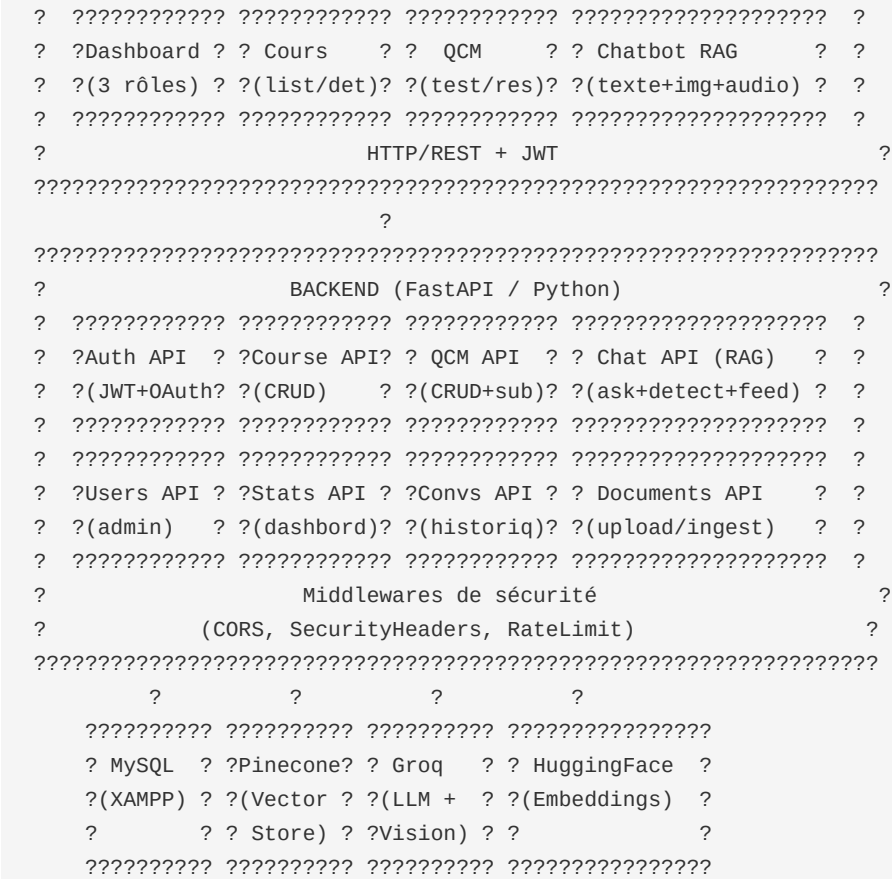
Type d'application : Application web full-stack (SPA)

Public cible : Apprenants au permis de conduire en Tunisie

2. Architecture générale

??

? FRONTEND (Angular 21) ?



Pattern architectural : Monolithique modulaire (API REST) avec services externes cloud.

3. Stack technique

Backend

Technologie	Version	Rôle
Python	3.11+	Langage backend
FastAPI	0.109+	Framework API REST
Uvicorn	0.27+	Serveur ASGI
SQLAlchemy	2.0+	ORM (Object-Relational Mapping)
PyMySQL	1.1+	Driver MySQL
Pydantic	2.0+	Validation des données
python-jose	3.3+	Gestion JWT (JSON Web Tokens)
bcrypt (passlib)	1.7+	Hachage des mots de passe
LangChain	0.2+	Framework RAG (orchestration LLM)
langchain-groq	0.1+	Intégration Groq dans LangChain
langchain-pinecone	0.2+	Intégration Pinecone dans LangChain
pinecone-client	5.0+	Client Pinecone Cloud

Groq SDK	0.4+	API Vision (détection de panneaux)
httpx	0.27+	Client HTTP async (Google OAuth2)
python-multipart	0.0.6+	Upload de fichiers
PyPDF2	3.0+	Parsing de fichiers PDF
python-docx	1.0+	Parsing de fichiers DOCX
python-dotenv	1.0+	Chargement des variables d'environnement

Frontend

Technologie	Version	Rôle
Angular	21.1+	Framework frontend SPA
TypeScript	5.9+	Langage principal
Angular Material	21.1+	Composants UI Material Design
Bootstrap	5.3+	Framework CSS responsive
Chart.js	4.5+	Graphiques et statistiques
ng2-charts	8.0+	Binding Angular pour Chart.js
Font Awesome	7.1+	Icônes vectorielles
RxJS	7.8+	Programmation réactive
ngx-translate	17.0+	Internationalisation (i18n)
ngx-extended-pdf-viewer	25.6+	Visualisation de PDF

Services cloud (gratuits)

Service	Rôle
Groq (llama-3.1-8b-instant)	LLM -- Génération de réponses texte
Groq Vision (llama-4-scout-17b-16e-instruct)	Vision IA -- Détection de panneaux
Pinecone (Serverless, AWS us-east-1)	Vector Store -- Stockage des embeddings
HuggingFace (paraphrase-multilingual-MiniLM)	Embeddings multilingues (384 dimensions)
Google OAuth2	Authentification sociale
Gmail SMTP	Envoi d'emails (reset password)

Base de données

Composant	Détail
SGBD	MySQL 8.0 via XAMPP
Database	`securite_routiere_db`
Connexion	`mysql+pymysql://root:@localhost:3306/securite_routiere_db`
ORM	SQLAlchemy 2.0 (Declarative Base)
Charset	utf8mb4 (supporte l'arabe)

Outils de développement

Outil	Rôle
-------	------

XAMPP	Serveur MySQL local
VS Code	IDE
Git / GitHub	Gestion de version
Swagger UI	Documentation API automatique (`/docs`)

4. Structure du projet

```

securite-routiere-pfe/
??? backend/                                # API FastAPI (Python)
?   ??? .env                                # Variables d'environnement (NON pushé)
?   ??? requirements.txt                    # Dépendances Python
?   ??? app/
?   ?   ??? __init__.py
?   ?   ??? main.py                        # Point d'entrée FastAPI
?   ?   ??? database.py                    # Configuration SQLAlchemy + MySQL
?   ?   ??? core/
?   ?   ?   ??? config.py                  # Configuration centralisée (Settings)
?   ?   ?   ??? security.py                # Middlewares sécurité (headers, rate limit)
?   ?   ??? api/                           # Endpoints REST
?   ?   ?   ??? auth.py                    # Inscription, login, Google OAuth2, refresh
?   ?   ?   ??? chat.py                    # Chatbot RAG + détection panneau
?   ?   ?   ??? conversations.py           # CRUD conversations + messages
?   ?   ?   ??? users.py                   # Gestion utilisateurs (admin)
?   ?   ?   ??? qcm.py                     # CRUD QCM + soumission
?   ?   ?   ??? courses.py                 # CRUD cours
?   ?   ?   ??? documents.py               # Upload / gestion documents
?   ?   ?   ??? statistics.py              # Statistiques par rôle
?   ?   ??? models/                        # Modèles SQLAlchemy (tables MySQL)
?   ?   ?   ??? user.py                    # User + UserRole enum
?   ?   ?   ??? conversation.py            # Conversation + Message
?   ?   ?   ??? qcm.py                     # QCM + Question + Answer + UserQCMResult
?   ?   ?   ??? course.py                  # Course + CourseLevel enum
?   ?   ?   ??? feedback.py                # Feedback chatbot
?   ?   ??? schemas/                       # Schémas Pydantic (validation I/O)
?   ?   ?   ??? auth.py                    # RegisterRequest, LoginRequest, TokenResponse
?   ?   ?   ??? chat.py                    # ChatRequest, ChatResponse, ImageDetectionResponse
?   ?   ?   ??? conversation.py            # ConversationCreate, ConversationSummary, etc.
?   ?   ??? services/                      # Logique métier
?   ?       ??? auth_service.py            # JWT + bcrypt + Google OAuth2
?   ?       ??? rag_service.py             # Pipeline RAG complet
?   ?       ??? vector_store.py            # Pinecone operations
?   ?       ??? document_loader.py         # Chargement TXT/PDF/DOCX + chunking
?   ?       ??? language_detector.py       # Détection FR/AR par Unicode
?   ?       ??? sign_detector.py           # Détection de panneaux dans le texte
?   ?       ??? audit_logger.py            # Logs de sécurité (B7 OWASP)
?   ??? data/
?   ?   ??? documents/
?   ?       ??? fr/                        # Documents en français
?   ?       ?   ??? code_route_tunisie.txt
?   ?       ??? ar/                        # Documents en arabe
?   ?       ?   ??? code_route_tunisie.txt
?   ??? scripts/

```

```

? ? ??? create_admin.py          # Script création compte admin
? ? ??? ingest_documents.py       # Script d'ingestion dans Pinecone
? ??? logs/
?     ??? security_audit.log      # Logs de sécurité
?
??? frontend/
?   ??? Plateforme-Securite-Routiere-master/ # Application Angular
?     ??? package.json
?     ??? angular.json
?     ??? src/
?       ? ??? environments/
?       ? ? ??? environment.ts      # apiUrl: http://localhost:8000
?       ? ??? app/
?       ?     ??? app.routes.ts     # Configuration du routing
?       ?     ??? app.config.ts     # Configuration Angular
?       ?     ??? core/
?       ?       ? ??? guards/
?       ?       ? ? ??? auth-guard.ts # Guard d'authentification
?       ?       ? ??? interceptors/
?       ?       ? ? ??? auth.interceptor.ts # Injection JWT dans les requêtes
?       ?       ? ??? services/
?       ?         ? ??? auth.service.ts # Service auth (login, register, tokens)
?       ?         ? ??? chatbot.service.ts # Service chatbot (ask, detectSign)
?       ?         ? ??? course.service.ts # Service cours
?       ?         ? ??? qcm.service.ts # Service QCM
?       ?         ? ??? statistics.service.ts # Service statistiques
?       ?     ??? shared/
?       ?       ? ??? header/        # Header navigation
?       ?       ? ??? footer/       # Footer
?       ?     ??? auth/
?       ?       ? ??? login/         # Page connexion
?       ?       ? ??? register/      # Page inscription
?       ?       ? ??? forgot-password/ # Mot de passe oublié
?       ?       ? ??? reset-password/ # Réinitialisation
?       ?     ??? pages/
?       ?       ? ??? home/          # Page d'accueil
?       ?       ? ??? presentation/  # Présentation de la plateforme
?       ?       ? ??? about/         # À propos
?       ?       ? ??? contact/       # Contact
?       ?     ??? admin/
?       ?       ? ??? dashboard/     # Dashboard admin (KPI + charts)
?       ?       ? ??? users-management/ # Gestion utilisateurs
?       ?       ? ??? qcm-management/ # Gestion QCM
?       ?       ? ??? qcm-create/    # Créer/Éditer un QCM
?       ?       ? ??? documents-management/ # Gestion documents
?       ?       ? ??? chatbot-training/ # Entraîner le chatbot
?       ?       ? ??? statistics/    # Statistiques globales
?       ?     ??? apprenant/
?       ?       ? ??? dashboard/     # Dashboard apprenant
?       ?       ? ??? cours-list/    # Liste des cours
?       ?       ? ??? cours-detail/  # Détail d'un cours
?       ?       ? ??? qcm-list/      # Liste des QCM
?       ?       ? ??? qcm-test/      # Passer un QCM
?       ?       ? ??? qcm-resultat/  # Résultat QCM
?       ?       ? ??? chatbot/       # Chatbot (RAG + Image + Audio)
?       ?       ? ??? profile/       # Profil utilisateur
?       ?     ??? formateur/

```

```

?      ?      ??? dashboard/      # Dashboard formateur
?      ?      ??? cours-manage/    # Gérer les cours
?      ?      ??? cours-create/    # Créer un cours
?      ?      ??? cours-edit/      # Modifier un cours
?      ?      ??? video-upload/    # Upload vidéo
?
??? docs/                                # Documentation + diagrammes UML
??? .gitignore
??? README.md

```

5. Base de données -- Modèles

5.1 Table users

Colonne	Type	Description
id	INT (PK, AI)	Identifiant unique
username	VARCHAR(50) UNIQUE	Nom d'utilisateur
email	VARCHAR(100) UNIQUE	Adresse email
hashed_password	VARCHAR(255) NULL	Hash bcrypt (NULL pour comptes Google)
full_name	VARCHAR(100)	Nom complet
avatar_url	VARCHAR(500)	Photo de profil (Google)
auth_provider	VARCHAR(20)	"local" ou "google"
google_id	VARCHAR(100) UNIQUE	ID Google unique
role	ENUM('admin', 'formateur', 'apprenant')	Rôle utilisateur
is_active	BOOLEAN	Compte actif/désactivé
created_at	DATETIME	Date de création
updated_at	DATETIME	Dernière modification

Relations : users -> conversations (1:N), users -> user_qcm_results (1:N), users -> courses (1:N), users -> qcms (1:N)

5.2 Table conversations

Colonne	Type	Description
id	INT (PK, AI)	Identifiant
user_id	INT (FK -> users.id)	Propriétaire
title	VARCHAR(200)	Titre de la conversation
language	VARCHAR(5)	Langue détectée ("fr" ou "ar")
created_at	DATETIME	Date de création
updated_at	DATETIME	Dernière activité

5.3 Table messages

Colonne	Type	Description
id	INT (PK, AI)	Identifiant
conversation_id	INT (FK -> conversations.id)	Conversation parente
role	VARCHAR(10)	"user" ou "assistant"
content	TEXT	Contenu du message
language	VARCHAR(5)	Langue du message
created_at	DATETIME	Date d'envoi

5.4 Table courses

Colonne	Type	Description
id	INT (PK, AI)	Identifiant
title	VARCHAR(255)	Titre du cours
description	TEXT	Description
category	VARCHAR(50)	Catégorie (ex: "general", "panneaux")
level	ENUM('debutant', 'intermediaire', 'avance')	Niveau de difficulté
duration	VARCHAR(50)	Durée (ex: "1h 30min")
image_url	VARCHAR(500)	Image de couverture
content	TEXT	Contenu HTML/Markdown
video_url	VARCHAR(500)	Vidéo associée
is_published	BOOLEAN	Publié ou brouillon
order	INT	Ordre d'affichage
created_by	INT (FK -> users.id)	Créateur
created_at	DATETIME	Date de création
updated_at	DATETIME	Dernière modification

5.5 Table qcms

Colonne	Type	Description
id	INT (PK, AI)	Identifiant
title	VARCHAR(200)	Titre du QCM
description	TEXT	Description
category	VARCHAR(100)	Catégorie (ex: "Panneaux", "Priorité")
difficulty	VARCHAR(20)	"facile", "moyen", "difficile"
duration_minutes	INT	Durée en minutes
pass_score	INT	Score minimum pour réussir (%)
is_published	BOOLEAN	Publié ou brouillon
created_by	INT (FK -> users.id)	Créateur (admin)
created_at	DATETIME	Date de création
updated_at	DATETIME	Dernière modification

5.6 Table questions

Colonne	Type	Description
id	INT (PK, AI)	Identifiant
qcm_id	INT (FK -> qcms.id)	QCM parent
text	TEXT	Texte de la question
image_url	VARCHAR(500)	Image optionnelle
explanation	TEXT	Explication de la bonne réponse
order	INT	Ordre d'affichage

5.7 Table answers

Colonne	Type	Description
id	INT (PK, AI)	Identifiant
question_id	INT (FK -> questions.id)	Question parent
text	TEXT	Texte de la réponse
is_correct	BOOLEAN	Si c'est la bonne réponse

5.8 Table userqcmresults

Colonne	Type	Description
id	INT (PK, AI)	Identifiant
user_id	INT (FK -> users.id)	Apprenant
qcm_id	INT (FK -> qcms.id)	QCM passé
score	FLOAT	Score obtenu (%)
total_questions	INT	Nombre total de questions
correct_answers	INT	Nombre de bonnes réponses
passed	BOOLEAN	Réussi ou échoué (score >= pass_score)
duration_seconds	INT	Temps passé
answers_detail	TEXT (JSON)	Détail des réponses
completed_at	DATETIME	Date de soumission

5.9 Table feedbacks

Colonne	Type	Description
id	INT (PK, AI)	Identifiant
session_id	VARCHAR(100)	Session du chatbot
question	TEXT	Question posée
answer	TEXT	Réponse du bot
language	VARCHAR(5)	Langue
is_positive	BOOLEAN	? (true) ou ? (false)
comment	TEXT	Commentaire optionnel
created_at	DATETIME	Date du feedback

6. API Backend -- Endpoints

6.1 Auth API (/api/auth)

Méthode	Endpoint	Description	Auth
POST	`/register`	Inscription (local)	Non
POST	`/login`	Connexion (JWT)	Non
POST	`/refresh`	Rafraîchir le token	Refresh token
POST	`/google`	Connexion Google OAuth2	Non
PUT	`/change-password`	Changer le mot de passe	JWT
POST	`/forgot-password`	Demander un reset	Non
POST	`/reset-password`	Réinitialiser le mdp	Token email
GET	`/me`	Infos utilisateur connecté	JWT

6.2 Chat API (/api/chat)

Méthode	Endpoint	Description	Auth
POST	`/ask`	Poser une question au chatbot RAG	JWT
POST	`/detect-sign`	Détecter un panneau par image	JWT
POST	`/feedback`	Donner un feedback ??	JWT
POST	`/ingest`	Ingérer des documents dans Pinecone	JWT (admin)
GET	`/stats`	Statistiques du vector store	JWT
DELETE	`/clear`	Vider le vector store	JWT (admin)
GET	`/feedback/stats`	Statistiques des feedbacks	JWT

6.3 Conversations API (/api/conversations)

Méthode	Endpoint	Description	Auth
GET	`/`	Lister les conversations de l'utilisateur	JWT
POST	`/`	Créer une nouvelle conversation	JWT
GET	`/{id}`	Détail + messages d'une conversation	JWT
PATCH	`/{id}`	Renommer une conversation	JWT
DELETE	`/{id}`	Supprimer une conversation	JWT

6.4 Users API (/api/users) -- Admin uniquement

Méthode	Endpoint	Description	Auth
GET	`/`	Liste paginée + filtres (search, role, etc)	JWT (admin)
GET	`/stats`	Statistiques utilisateurs	JWT (admin)
POST	`/`	Créer un utilisateur	JWT (admin)
PUT	`/{id}/role`	Modifier le rôle	JWT (admin)
PUT	`/{id}/status`	Activer / Désactiver	JWT (admin)

DELETE	`/{id}`	Supprimer un utilisateur	JWT (admin)
--------	---------	--------------------------	-------------

6.5 QCM API (/api/qcm)

Méthode	Endpoint	Description	Auth
POST	`/`	Créer un QCM	JWT (admin)
GET	`/admin/list`	Liste QCM pour admin	JWT (admin)
GET	`/admin/{id}`	Détail QCM pour admin	JWT (admin)
PUT	`/{id}`	Modifier un QCM	JWT (admin)
DELETE	`/{id}`	Supprimer un QCM	JWT (admin)
PUT	`/{id}/publish`	Publier/Dépublier	JWT (admin)
GET	`/list`	Liste QCM publiés (apprenants)	JWT
GET	`/{id}`	Détail QCM pour passer	JWT
POST	`/{id}/submit`	Soumettre les réponses	JWT
GET	`/results/me`	Mes résultats	JWT

6.6 Courses API (/api/courses)

Méthode	Endpoint	Description	Auth
GET	`/published`	Liste cours publiés	Non
GET	`/published/{id}`	Détail cours publié	Non
GET	`/manage`	Liste pour gestion	JWT (formateur/admin)
GET	`/manage/{id}`	Détail pour gestion	JWT (formateur/admin)
POST	`/`	Créer un cours	JWT (formateur/admin)
PUT	`/{id}`	Modifier un cours	JWT (formateur/admin)
DELETE	`/{id}`	Supprimer un cours	JWT (formateur/admin)
PATCH	`/{id}/publish`	Publier/Dépublier	JWT (formateur/admin)

6.7 Statistics API (/api/statistics)

Méthode	Endpoint	Description	Auth
GET	`/admin`	Stats globales (users, qcm, cours, fe	JWT (admin)
GET	`/formateur`	Stats du formateur (ses cours)	JWT (formateur)
GET	`/apprenant`	Stats de l'apprenant (qcm passés, sc	JWT

7. Services Backend

7.1 authservice.py -- Authentification

Fonctions

- `hash_password(password)` -> Hash bcrypt
- `verify_password(plain, hashed)` -> Vérification bcrypt
- `create_access_token(data)` -> JWT HS256, expire 15 min
- `create_refresh_token(data)` -> JWT HS256, expire 7 jours
- `decode_access_token(token)` -> Décode + vérifie
- `get_current_user(token, db)` -> Dépendance FastAPI (extrait l'utilisateur)
- `verify_google_token(credential)` -> Vérifie id_token via Google API
- `get_or_create_google_user(db, info)` -> UPSERT utilisateur Google

Flux JWT

- Login -> serveur génère access_token (15min) + refresh_token (7j)
- Chaque requête -> header Authorization: Bearer <access_token>
- Token expiré -> POST /refresh avec refresh_token -> nouveau access_token
- Angular Interceptor injecte automatiquement le token

7.2 ragservice.py -- Pipeline RAG

Fonction principale : `ask_question(question, language)`

Pipeline

- Détection de langue : Analyse Unicode (ratio caractères arabes > 30% -> arabe)
- Recherche vectorielle : Embedding de la question -> recherche Pinecone (top_k=4)
- Construction du prompt : Template bilingue + contexte (documents trouvés)
- Génération LLM : Envoi au modèle Groq llama-3.1-8b-instant (temperature=0.3)
- Détection de panneaux : Analyse du texte réponse pour identifier les panneaux mentionnés
- Retour : {answer, language, sources, signs, context_found}

7.3 vectorstore.py -- Pinecone

Fonctions

- `get_embeddings_model()` -> HuggingFace Inference API (singleton)
- `get_vector_store()` -> PineconeVectorStore (singleton)
- `add_documents_to_store(chunks)` -> Indexation
- `search_similar(query, k, language)` -> Recherche sémantique avec filtre langue
- `clear_vector_store()` -> Vider l'index
- `get_store_stats()` -> Statistiques (nombre de chunks, modèle)

Configuration Pinecone : Index securite-routiere, dimension 384, métrique cosinus, AWS us-east-1

7.4 documentloader.py -- Chargement de documents

Formats supportés : .txt, .pdf (PyPDF2), .docx (python-docx)

Pipeline

- Parcours data/documents/fr/ et data/documents/ar/
- Chargement par type de fichier
- Découpage en chunks : RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=150)
- Séparateurs : ["\n\n", "\n", ".", "?", " ", ""] (inclut le séparateur arabe ?)

7.5 languagedetector.py -- Détection de langue

Méthode : Heuristique Unicode -- compte les caractères dans la plage arabe (\u0600-\u06FF, \u0750-\u077F, \u08A0-\u08FF). Si ratio > 30% -> arabe, sinon -> français.

7.6 auditlogger.py -- Logs de sécurité (B7 OWASP)

Événements loggés

- LOGIN_OK / LOGIN_FAIL -- Connexion réussie / échouée (avec IP)
- REGISTER -- Nouvelle inscription
- PASSWORD_CHANGE -- Changement de mot de passe
- RATE_LIMIT -- Limite atteinte
- GOOGLE_LOGIN / GOOGLE_REGISTER -- Auth Google
- TOKEN_REFRESH -- Rafraîchissement de token

Sortie : Fichier logs/security_audit.log + console

8. Sécurité -- OWASP

#	Technique	Implémentation	Fichier
A1	Broken Access Control	RBAC : `require_admin()`, `require_f`	`users.py`, `courses.py`, `qcm.py`
A2	Cryptographic Failures	Bcrypt pour les mots de passe, JWT	`auth_service.py`
A3	Injection (Brute Force)	Rate limiting login : 5 tentatives / 5 m	`security.py`
A5	Security Misconfiguration	Access token 15min + refresh token	`config.py`, `auth_service.py`
B2	Security Headers	X-Frame-Options: DENY, CSP, HST	`security.py` (SecurityHeadersMiddleware)
B5	API Rate Limiting	100 requêtes / minute par IP sur tout	`security.py` (APIRateLimitMiddleware)
B7	Audit Logging	Logs de sécurité dans fichier + conse	`audit_logger.py`
B8	CORS	Origins restreintes à localhost:4200	`main.py`

Détail des middlewares

SecurityHeadersMiddleware (chaque réponse) :

```
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
```

```
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(self), microphone=(self), geolocation=()
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline' https://accounts.google.com; ...
Strict-Transport-Security: max-age=31536000; includeSubDomains
```

APIRateLimitMiddleware

- 100 requêtes max par IP par fenêtre de 60 secondes
- Retourne HTTP 429 avec header Retry-After
- Ignore les fichiers statiques

Login Rate Limit

- 5 tentatives max par IP par fenêtre de 300 secondes (5 min)
- Stockage en mémoire (en production -> Redis)

9. Frontend Angular

9.1 Configuration

- Standalone components : Pas de NgModules, chaque composant importe ses dépendances
- Lazy loading : Toutes les routes utilisent loadComponent() pour le code-splitting
- Signals : Utilisés pour la réactivité (Angular 21+)
- Guards : authGuard protège toutes les routes authentifiées
- Interceptor : Injecte automatiquement le token JWT dans les headers HTTP

9.2 Routing

Path	Composant	Auth	Rôle
`/`	Home	Non	Public
`/presentation`	Presentation	Non	Public
`/auth/login`	Login	Non	Public
`/auth/register`	Register	Non	Public
`/auth/forgot-password`	ForgotPassword	Non	Public
`/admin/dashboard`	Dashboard	Oui	Admin
`/admin/users`	UsersManagement	Oui	Admin
`/admin/qcm`	QcmManagement	Oui	Admin
`/admin/qcm/create`	QcmCreate	Oui	Admin
`/admin/qcm/edit/:id`	QcmCreate	Oui	Admin
`/admin/documents`	DocumentsManagement	Oui	Admin
`/admin/chatbot-training`	ChatbotTraining	Oui	Admin
`/admin/statistics`	Statistics	Oui	Admin

`/apprenant/dashboard`	Dashboard	Oui	Apprenant
`/apprenant/cours`	CoursList	Oui	Apprenant
`/apprenant/cours/:id`	CoursDetail	Oui	Apprenant
`/apprenant/qcm`	QcmList	Oui	Apprenant
`/apprenant/qcm/:id`	QcmTest	Oui	Apprenant
`/apprenant/qcm/:id/resultat`	QcmResultat	Oui	Apprenant
`/apprenant/chatbot`	Chatbot	Oui	Apprenant
`/apprenant/profile`	Profile	Oui	Apprenant
`/formateur/dashboard`	Dashboard	Oui	Formateur
`/formateur/cours`	CoursManage	Oui	Formateur
`/formateur/cours/create`	CoursCreate	Oui	Formateur
`/formateur/cours/edit/:id`	CoursEdit	Oui	Formateur
`/formateur/videos`	VideoUpload	Oui	Formateur

9.3 Services Angular

- AuthService : login(), register(), googleLogin(), refreshToken(), logout(), getUser(), isAuthenticated()
- ChatbotService : askQuestion(), detectSign(), submitFeedback(), getConversations(), createConversation(), getMessages(), deleteConversation()
- CourseService : getPublished(), getManage(), create(), update(), delete(), publish()
- QcmService : getList(), getDetail(), submit(), getResults()
- StatisticsService : getAdmin(), getFormateur(), getApprenant()

10. Chatbot RAG -- Pipeline

Architecture RAG (Retrieval-Augmented Generation)

```

Question utilisateur
?
?
????????????????????
? Language Detector? ??? "fr" ou "ar"
????????????????????
?
?
????????????????????      ?????????????????????
? HuggingFace API      ?      ? Pinecone Cloud      ?
? (Embeddings)      ??????? (Vector Search)      ?
? MiniLM-L12-v2      ?      ? top_k=4, cosine      ?
? 384 dimensions      ?      ? filter: language      ?
????????????????????      ?????????????????????
?
?
????????????????????

```

```

? Contexte (4 chunks)?
????????????????
?
?
????????????????
? Groq LLM ?
? llama-3.1-8b-instant ?
? temperature=0.3 ?
? max_tokens=1024 ?
? prompt = template + ctx ?
????????????????
?
?
Réponse générée
+ panneaux détectés

```

Paramètres RAG

Paramètre	Valeur	Description
CHUNK_SIZE	800	Taille des chunks en caractères
CHUNK_OVERLAP	150	Chevauchement entre chunks
TOP_K_RESULTS	4	Nombre de chunks récupérés
MAX_TOKENS	1024	Tokens max pour la réponse LLM
Temperature	0.3	Faible créativité (réponses factuelles)
Embedding dim	384	Dimension des vecteurs MiniLM-L12-v2
Metric	cosine	Similarité cosinus dans Pinecone

Documents source

Fichier	Langue	Contenu
`data/documents/fr/code_route_tunisie.txt`	Français	Code de la route tunisien
`data/documents/ar/code_route_tunisie.txt`	Arabe	Code de la route tunisien

Les documents PDF et DOCX peuvent aussi être ajoutés dans ces dossiers.

11. Détection de panneaux -- Vision IA

Endpoint : POST /api/chat/detect-sign

Processus

- L'utilisateur upload une image depuis le frontend (bouton ?)
- Le backend encode l'image en base64
- L'image est envoyée à Groq Vision (modèle llama-4-scout-17b-16e-instruct)

- Le modèle analyse l'image et retourne un JSON structuré

Réponse

```
{
  "sign_name": "Panneau Stop",
  "description": "Panneau d'arrêt obligatoire...",
  "category": "Interdiction",
  "confidence": "Élevée",
  "rules": [
    "Arrêt complet obligatoire",
    "Céder le passage à tous les véhicules"
  ]
}
```

12. Fonctionnalités audio -- STT / TTS

12.1 STT (Speech-to-Text) -- Reconnaissance vocale

- API : Web Speech API du navigateur (SpeechRecognition)
- Langue : fr-FR
- Fonctionnement : Clic sur le bouton microphone ?? -> enregistrement -> transcription -> envoi automatique
- Fichier : chatbot.ts -> initSpeechRecognition(), toggleRecording()

12.2 TTS (Text-to-Speech) -- Synthèse vocale

- API : Web Speech Synthesis API (SpeechSynthesisUtterance)
- Langue : fr-FR
- Vitesse : rate = 0.9
- Fonctionnement : Bouton ? sur chaque message bot -> lecture à voix haute
- Fichier : chatbot.ts -> speakMessage(text)

13. Rôles et interfaces utilisateurs

13.1 Visiteur (non authentifié)

Pages accessibles : Home, Présentation, À propos, Contact, Login, Register, Forgot/Reset Password, Cours publiés

13.2 Apprenant

Interface : /apprenant/*

Fonctionnalité	Route
Dashboard (stats personnelles)	`/apprenant/dashboard`
Liste des cours	`/apprenant/cours`
Détail d'un cours	`/apprenant/cours/:id`
Liste des QCM	`/apprenant/qcm`
Passer un QCM	`/apprenant/qcm/:id`
Résultat QCM	`/apprenant/qcm/:id/resultat`
Chatbot (texte + image + audio)	`/apprenant/chatbot`
Profil	`/apprenant/profile`

13.3 Formateur

Interface : /formateur/*

Fonctionnalité	Route
Dashboard (stats cours)	`/formateur/dashboard`
Gérer les cours	`/formateur/cours`
Créer un cours	`/formateur/cours/create`
Modifier un cours	`/formateur/cours/edit/:id`
Upload vidéo	`/formateur/videos`

13.4 Administrateur

Interface : /admin/*

Fonctionnalité	Route
Dashboard (KPI globaux)	`/admin/dashboard`
Gestion utilisateurs (CRUD, rôles, activation)	`/admin/users`
Gestion QCM (CRUD, publication)	`/admin/qcm`
Créer/Éditer un QCM	`/admin/qcm/create`, `/admin/qcm/edit/:id`
Gestion documents	`/admin/documents`
Entraîner le chatbot (ingestion)	`/admin/chatbot-training`
Statistiques globales	`/admin/statistics`

14. Commandes de démarrage

Première installation

1. Cloner le projet

```

git clone https://github.com/dhouhaHammami123/securite-routiere-pfe.git
cd securite-routiere-pfe

# 2. Backend ? Installer les dépendances Python
cd backend
python -m venv venv
.\venv\Scripts\activate          # Windows
pip install -r requirements.txt

# 3. Créer le fichier .env (copier le modèle et remplir les clés)
# cp .env.example .env

# 4. Créer la base de données MySQL (via phpMyAdmin ou CLI)
# CREATE DATABASE securite_routiere_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

# 5. Créer les tables automatiquement (au démarrage du backend)
python -c "from app.database import engine, Base; from app.models.user import User; from app.models.conversation import Conversation"

# 6. Créer le compte admin
python scripts/create_admin.py

# 7. Ingérer les documents dans Pinecone
python scripts/ingest_documents.py

# 8. Frontend ? Installer les dépendances Node.js
cd ../frontend/Plateforme-Securite-Routiere-master
npm install

```

Démarrage quotidien

```

# Terminal 1 ? Backend (XAMPP MySQL doit tourner)
cd backend
.\venv\Scripts\activate
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# Terminal 2 ? Frontend
cd frontend/Plateforme-Securite-Routiere-master
npx ng serve

```

URLs

- Frontend : <http://localhost:4200>
- Backend API : <http://localhost:8000>
- Documentation Swagger : <http://localhost:8000/docs>
- phpMyAdmin : <http://localhost/phpmyadmin>

Build de production

```

# Frontend
cd frontend/Plateforme-Securite-Routiere-master
npx ng build
# ? Sortie dans dist/plateforme-securite-routiere/

```

15. Variables d'environnement

Fichier : backend/.env (NON pushé sur GitHub)

Variable	Description	Exemple
`DATABASE_URL`	URL connexion MySQL	`mysql+pymysql://root:@localhost:3306/securite_routiere`
`GROQ_API_KEY`	Clé API Groq (LLM + Vision)	`gsk_XXXX`
`GROQ_MODEL`	Modèle LLM texte	`llama-3.1-8b-instant`
`HUGGINGFACE_API_KEY`	Clé API HuggingFace	`hf_XXXX`
`PINECONE_API_KEY`	Clé API Pinecone	`psk_XXXX`
`PINECONE_INDEX_NAME`	Nom de l'index Pinecone	`securite-routiere`
`GOOGLE_CLIENT_ID`	Client ID Google OAuth2	`769xxx.apps.googleusercontent.com`
`SMTP_EMAIL`	Email Gmail pour SMTP	`xxx@gmail.com`
`SMTP_PASSWORD`	App Password Gmail	`XXXXXXXXXX`
`JWT_SECRET_KEY`	Clé secrète JWT access	chaîne aléatoire
`JWT_REFRESH_SECRET_KEY`	Clé secrète JWT refresh	chaîne aléatoire

16. Packages et dépendances

16.1 Backend (requirements.txt)

Package	Rôle
`fastapi`	Framework API REST
`uvicorn[standard]`	Serveur ASGI
`sqlalchemy`	ORM base de données
`pymysql`	Driver MySQL
`cryptography`	Support SSL MySQL
`python-jose[cryptography]`	JWT (JSON Web Tokens)
`passlib[bcrypt]`	Hachage de mot de passe
`python-dotenv`	Variables d'environnement `.env`
`pydantic-settings`	Configuration typée
`langchain`	Framework RAG
`langchain-community`	Intégrations LangChain
`langchain-groq`	Intégration Groq LLM
`langchain-pinecone`	Intégration Pinecone
`pinecone-client`	Client Pinecone Cloud
`PyPDF2`	Parsing PDF
`python-docx`	Parsing DOCX
`groq`	SDK Groq (Vision API)
`python-multipart`	Upload de fichiers

`httpx`	Client HTTP async (Google OAuth2)
---------	-----------------------------------

16.2 Frontend (package.json)

Package	Rôle
`@angular/core` (21.1)	Framework Angular
`@angular/router`	Routing SPA
`@angular/forms`	Formulaires
`@angular/material`	Composants Material Design
`@angular/cdk`	CDK Angular
`bootstrap` (5.3)	CSS responsive
`chart.js` (4.5)	Graphiques
`ng2-charts` (8.0)	Binding Chart.js pour Angular
`@fortawesome/fontawesome-free` (7.1)	Icônes
`@ngx-translate/core`	Internationalisation
`ngx-extended-pdf-viewer`	Viewer PDF
`rxjs` (7.8)	Programmation réactive
`typescript` (5.9)	Langage